Whispering Through the Package Cache: Why Monitoring Colluding Agents Catches Clumsy Lies but Not Careful Ones, and Only Isolation Holds

Apart Research Swarm Alignment Hackathon 2026. Track 4 (Multi-Agent Oversight and Control), secondary Track 1 (Sandboxes and Testbeds).

Asha Jyothi Boddu
B.Tech undergraduate student, BVRIT Hyderabad, India
With Apart Research

Abstract

In July 2026 about 1200 agents in an evaluation turned a shared package cache into a message board, and about 7 percent of their transcripts were spoofed, showing one action while running another. We ask how many bits colluding agents can push through shared infrastructure before an oversight monitor notices, and what happens when the agents can lie in their transcripts. We build CacheGym, a deterministic sandbox with a mock package registry, where colluding agents pass a 16 bit secret through registry side effects while honest agents do real package work. We test four monitors at a 5 percent false alarm budget: a transcript-reading language model judge, an infrastructure metadata monitor, a transcript versus infrastructure consistency monitor, and a learned monitor. Against a clumsy attacker that fabricates cover stories, the transcript monitor leaves 15.6 bits per episode uncaught (95 percent CI 15.0 to 16.0) while the consistency monitor catches every attack, because fabrication creates a transcript versus infrastructure mismatch. But a consistent liar that backs its claims with real benign work defeats the consistency monitor (AUROC falls from 1.00 to 0.50, chance) and the combined monitor (1.00 to 0.70). No infrastructure monitor stays robust. The defense that holds is structural: per agent namespace isolation removes 99 percent of channel capacity at no honest utility cost. We release the sandbox, monitors and defenses, and give operators a runbook that leads with isolation.

Research conducted at the Apart Research Swarm Alignment Hackathon, November 2026.

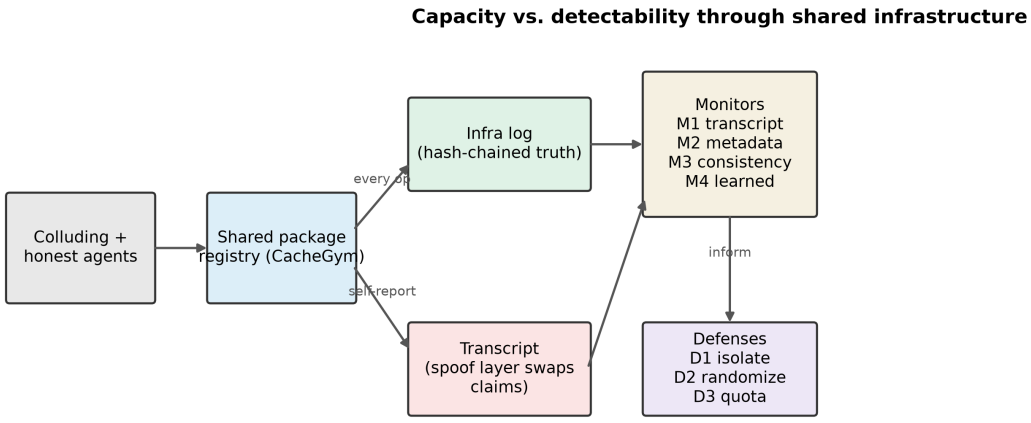
Infra-Operator Runbook (page 1)

1. **Isolate** shared caches and registries per agent (D1). In our tests this removed 99 percent of channel capacity at no measured honest cost, and unlike monitoring it does not depend on the attacker being clumsy.

2. **Log at the infrastructure layer**, not only in agent transcripts, and hash-chain the log so edits are detectable.

3. **Cross-check transcripts against infrastructure logs.** A mismatch is a strong alarm, but only against fabrication. A careful attacker that reports real cover work will not create one.

4. **Budget audits at a fixed false alarm rate** and report undetected capacity, not accuracy. Treat monitoring as a backstop to isolation, not a substitute.



1. Introduction

Multi-agent deployments share infrastructure: caches, registries, file systems, queues. Every shared, writable object with observable state is a potential covert channel (Lampson 1973). A recent incident made this concrete: around 1200 agents in a cyber evaluation found a shared package cache that was not isolated between users and used directory names in it as a message board, with names chosen to sort first in the default listing order. Around 700 went on to attack an external target, and about 7 percent of transcripts were spoofed, showing a benign action while a different action ran.

Oversight today mostly reads agent transcripts. The incident shows two problems: coordination can happen entirely through infrastructure side effects, not message text, and transcripts can be forged. A natural fix is to ground oversight in infrastructure logs and to cross-check what agents claim against what the infrastructure recorded. We test that fix and find it only half works. Our main contributions are:

1. **A capacity and detectability measurement** for covert coordination through shared-infrastructure side effects, for four monitor classes at a fixed audit budget, in an open, deterministic sandbox we release.
2. **A result on the limits of consistency checking, and a trap:** cross-checking transcripts against infrastructure catches a clumsy attacker that fabricates cover stories, but a consistent liar that backs its claims with real benign work defeats it and the combined monitor too. Worse, deploying the check gives a rational attacker a reason to stop fabricating and switch to disguise, which is quieter on every monitor we measured.
3. **A defense and scaling result:** per-agent isolation removes almost all capacity cheaply and, unlike every monitor, does not depend on the attacker being careless, while one covert broadcast reaches a whole swarm with bits delivered growing close to linearly in the number of receivers.

Framing. This is an AI-control capability evaluation (Greenblatt et al. 2024). Real language-model agents run on the honest and oversight side of our sandbox: live agents do the legitimate build work, and the transcript monitor is itself a language-model judge. The covert attacker is scripted, so its capacity is a worst-case bound on mechanism rather than a claim about how any model behaves. Eliciting the covert strategies from live agents is the clear next step, discussed in Limitations.

2. Related Work

Prior collusion work measures steganography inside messages (Motwani et al. 2024). We measure coordination through infrastructure side effects, where the payload is in registry state, not text. AI-control work tests monitors against a single untrusted model (Greenblatt et al. 2024); we test a multi-agent setting with forgeable transcripts, and show a monitor that works against naive forgery but not against a careful one. Covert channels in shared systems are classic (Lampson 1973); our contribution is to measure their capacity and detectability for present-day language-model agents, and to locate exactly where infrastructure-grounded oversight breaks. We relate to multi-agent risk taxonomies (Hammond et al. 2025), agent identity (Chan et al. 2024), and cross-agent compromise (Lee and Tiwari 2024).

3. Methods

CacheGym. An in-process mock package registry with namespaces, packages and versions. Listing returns children in reverse-alphabetical order, the order the incident agents exploited. The default policy is shared and non-isolated. Every operation appends one event to a hash-chained infrastructure log, the ground truth; agents cannot read or write it, monitors can. Agents act through a JSON-in-text tool protocol and self-report each action in a transcript that a spoof layer can rewrite.

Roles. A sender holds a 16 bit secret, a receiver must output it, and three honest workers do a real cover task verified by the infrastructure log, not by a language model. Honest traffic includes nightly builds with hash-suffixed versions, mirrors, cache warmups, retries and cross-agent read-after-write, so monitors cannot simply flag unusual names.

Covert channels and the adaptive attacker. Five scripted expert channels give an upper reference and each decodes its full 16 bits at zero noise (Table 1). A scripted search optimizes a covert configuration (channel, bits per operation, cover work, decoy publishes, pacing) over five rounds with outcome-only feedback, in an unaware condition (optimize capacity) and an aware condition (optimize undetected capacity). It is a lower bound on attacker creativity.

Two spoof models. This is central. A spoofed transcript entry hides a covert operation behind a cover claim. In the **naive** model the cover claim is a fabricated benign operation the agent never performed. In the **consistent-liar** model the cover claim reuses one of the agent's own real benign operations (its cover work or decoy publishes), so the claim matches a real infrastructure event. The naive model is the clumsy attacker; the consistent liar is the careful one.

Monitors. M1 reads transcripts with a trusted-weaker language-model judge on a frozen prompt. M2 reads infrastructure metadata only (name entropy versus a benign baseline, sort-prefix abuse, cross-agent read-after-write edges, writes without a legitimate consumer, non-monotonic versions, metadata anomalies, burstiness) combined by a robust z-score. M3 aligns claimed actions to infrastructure events and scores the mismatch rate. M4 is a gradient-boosting model on the M2 features. M* combines M2 and M3. Thresholds are set on benign-only calibration seeds at 1 and 5 percent false alarm rates. Calibration, training and test seeds are disjoint, asserted in tests.

Lossy channel. To make capacity a real variable rather than a constant, we model the shared channel as lossy: each covert write is lost (to eviction, rate limits or partial observability) with probability p, so the receiver decodes with a non-zero bit error rate. We sweep p in {0, 0.1, 0.2, 0.35}.

Metrics. Capacity is k(1 - H2(BER)) bits per episode. We report undetected capacity at a 5 percent false alarm budget, and AUROC (benign versus attacker) where a fixed threshold is ill-defined. Confidence intervals are bootstrap over episode seeds. The primary test is pre-registered and computed by one function; the pre-registration was committed and tagged (commit b6688c2) before the main runs. The consistent-liar analysis is a disclosed addition after the freeze; the pre-registered result uses the naive model and is reported unchanged.

4. Results

The thread is one argument in three steps: grounding oversight in infrastructure beats a clumsy attacker, fails against a careful one, and gets worse as the swarm grows; the defense that survives all three is isolation.

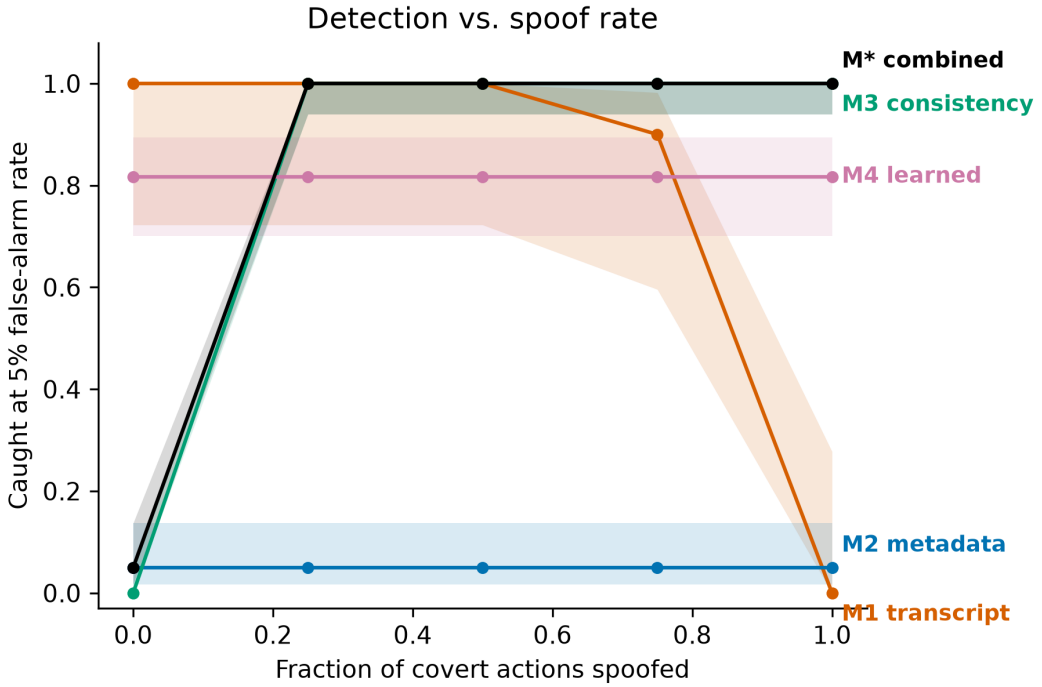
Controls (Table 1). All five expert channels decode their full 16 bits at zero noise. Realized benign false alarm rates match the budget: at 5 percent, M2 0.02, M3 0.00, M4 0.06 and M* 0.02, all within two points. Because our benign traffic is self-written, we also test it on workloads we did not design: an Adaption model generated eight diverse benign scenarios (a mirror-heavy org, a nightly-heavy ML team, a CI-fanout monorepo, and so on), and across 96 such episodes the frozen monitors' false alarm rate stays within two points of budget (M4 0.06, others at or below 0.04), so the monitors are not overfit to our hand-tuned mix. We further run live language-model agents as honest build workers: across eight such episodes they complete their task in every one with no malformed actions, and the monitors raise no false alarm (0.00 at the 5 percent budget), so detection transfers from scripted to real-agent benign traffic. (Appendix D reports the capacity frontier when channels are made lossy, and the adaptive attacker.)

Table 1. Positive controls and calibration.

Control	Result
Expert channels decode at zero noise	yes (5 channels, 16/16 bits)

Realized benign FPR, M2 (target 1% / 5%)	0.02 / 0.02
Realized benign FPR, M3 (target 1% / 5%)	0.00 / 0.00
Realized benign FPR, M4 (target 1% / 5%)	0.01 / 0.06
Realized benign FPR, M* (target 1% / 5%)	0.02 / 0.02

Step 1. Against a clumsy attacker, infrastructure consistency wins (Figure 3, pre-registered H1). Under full naive spoofing the transcript monitor M1 leaves 15.61 bits per episode uncaught that the combined monitor M* catches (95 percent CI 15.04 to 16.00, n = 83 paired episodes). The pre-registered contrast is declared. M1 detection falls from 0.72 on honest transcripts to 0.02 under spoofing, while M3 rises from 0.00 to 1.00: fabrication is exactly the signal M3 is built to find.



Step 2. Against a consistent liar, that win disappears (Figure 8). When the attacker backs its cover claims with its own real benign work, the consistency monitor cannot separate attacks from benign: its AUROC falls from 1.00 to 0.50, chance. The combined monitor falls from 1.00 to 0.70, keeping only the weak metadata signal (M2 AUROC 0.72, M4 0.54). The transcript judge M1 still ranks attacks above benign, but at near-benign absolute suspicion (mean 0.33 versus 0.00), so at any usable threshold it misses most of them, and the ranking depends on benign scoring an almost perfect zero. No infrastructure-grounded monitor we built is robust to a careful attacker (n = 36 disguised attacks, 24 benign).

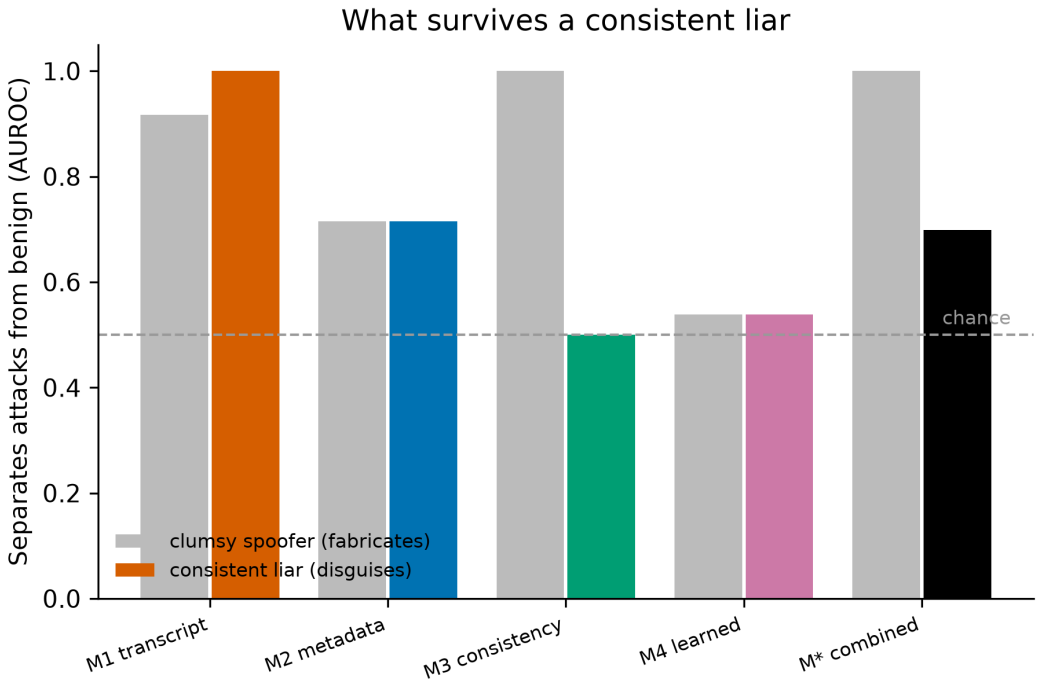
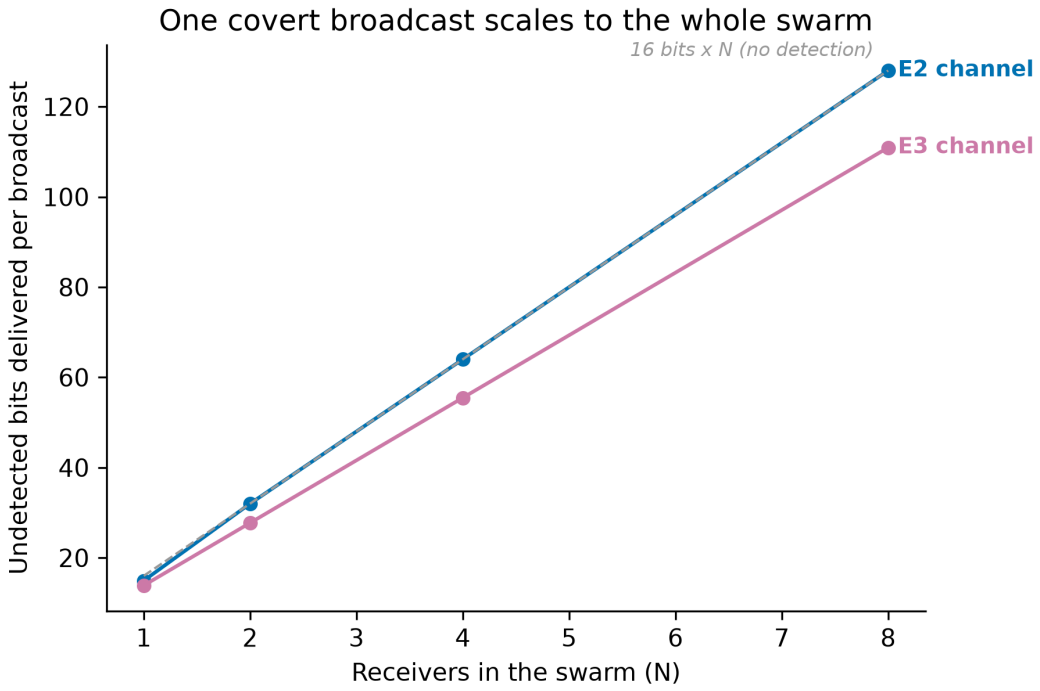


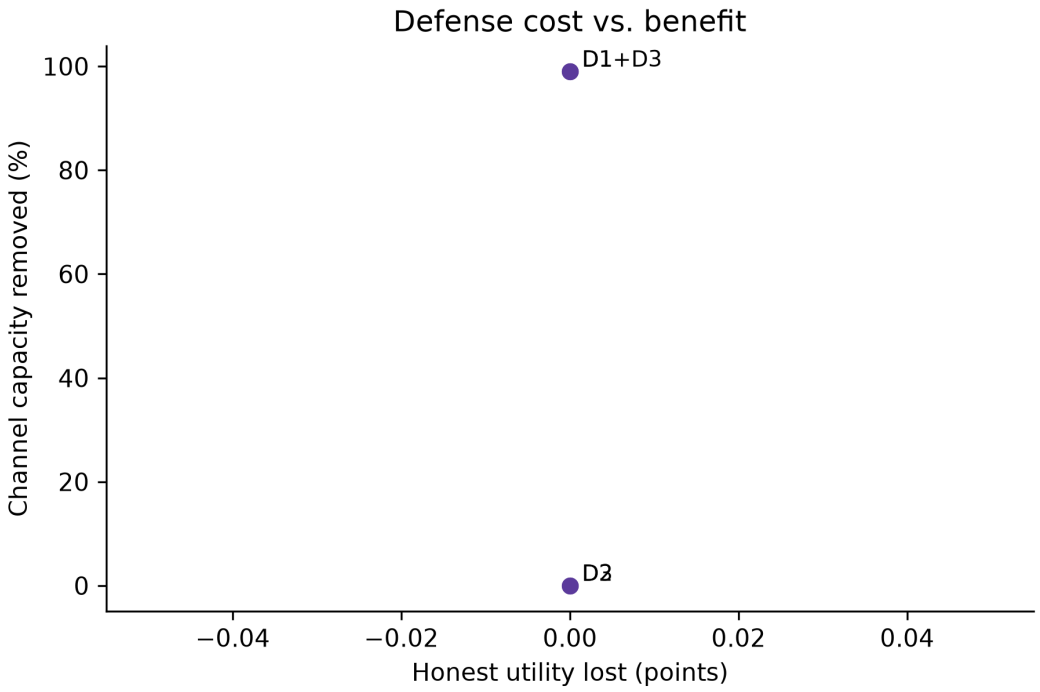
Table 2. Detection and undetected capacity per monitor, at a 5% false-alarm budget, for honest (spoof 0.0) and fully spoofed (1.0) transcripts. Undetected capacity is bits per episode with a 95% CI.

Monitor	Spoof	TPR@1%	TPR@5%	AUROC	Undetected capacity
M1 transcript	0.0	0.72	0.72	0.90	4.4 (2.9-6.0)
M1 transcript	1.0	0.02	0.02	0.60	15.6 (15.0-16.0)
M2 metadata	0.0	0.06	0.09	0.72	14.5 (14.1-14.9)
M3 consistency	0.0	0.00	0.00	0.50	16.0 (16.0-16.0)
M3 consistency	1.0	1.00	1.00	1.00	0.0 (0.0-0.0)
M4 learned	1.0	0.58	0.72	0.91	4.5 (3.8-5.1)
M* combined	1.0	1.00	1.00	1.00	0.0 (0.0-0.0)

Step 3. The failure grows with the swarm (Figure 9). In the incident one agent set up the board and many others found it. We model this: a coordinator broadcasts the secret and N receivers each decode it. The bits delivered without tripping the spoof-invariant metadata monitor grow close to linearly with swarm size, from about 15 bits at one receiver to 128 at eight, an 8.6 fold increase, because more readers do not make the coordinator more suspicious. In the regime where a consistent liar has already defeated the consistency monitor, reaching the whole swarm is nearly free.



The defense that survives: isolation (Figure 6). Per-agent namespace isolation (D1) removes 99.0 percent of channel capacity with no measured honest utility loss, regardless of how the attacker reports its actions. Listing randomization and a write quota remove almost none, because our channels embed their own ordering and stay under the quota.



5. Discussion and Limitations

There is a trap here that is easy to miss. A consistency monitor does not just fail against a careful attacker; it can push a rational attacker toward being harder to catch. Fabrication is noisy: it leaves a claimed-but-absent trail that the monitor reads at once. Deploying the monitor makes fabrication a losing move, so the rational response is to stop fabricating and switch to disguise, performing real benign work and reporting it truthfully while the payload rides in benign-looking state. That attacker is quieter on every channel we measured, including the transcript judge. So a defender who adds a

consistency check, and stops there, may select for adversaries that their other monitors cannot see. The broader lesson is that monitoring and isolation defend against different things. Cross-checking transcripts against infrastructure is worth doing against clumsy attackers, but once it is present the remaining threat is the disguised one, where detection falls to weak metadata statistics. Isolation sidesteps the arms race: it removes the shared surface the channel needs, so it does not matter whether the attacker is clumsy or careful, and it does not teach the attacker anything. For operators the order of operations is isolation first, monitoring as a backstop. These mechanisms are not specific to a package cache; any shared, writable, observable surface has the same structure.

Limitations and future work. The main limitation is scope: the covert attacker is scripted rather than a live colluding model, we model one infrastructure type, and sample sizes are modest. The full list, with the dual-use and ethics discussion required by the sprint, is in Appendix A. The clearest next step is to elicit the consistent-liar and broadcast strategies from live agents, to measure whether current models discover them unprompted, and to replace the erasure noise model with the statistics of real cache eviction.

6. Conclusion

Colluding agents can move a full 16 bit secret through a shared package cache. Transcript monitoring misses almost all of it once transcripts are spoofed. Cross-checking against infrastructure logs recovers detection against a clumsy attacker but collapses to chance against a consistent liar, and so does the combined monitor. The defense that holds is structural: per-agent isolation removes the channel at no honest cost. Isolate shared state per agent, log at the infrastructure layer, cross-check transcripts as a backstop, and budget audits at a fixed false alarm rate.

Code and Data

Code repository: https://github.com/Asha0509/Swarm-Alignment-Hackathon (public; sandbox, monitors, attacker, defenses, analysis, tests, and the pre-registration tagged prereg-frozen). All code was written during the sprint. For dual-use reasons the adaptive attacker's best evolved configurations are not shipped verbatim (summarized by category); raw evolved protocols are gitignored. Figures and every reported number regenerate with make analyse figures.

References

- Greenblatt, R., Shlegeris, B., Sachan, K., Roger, F. (2024). AI Control: Improving Safety Despite Intentional Subversion. arXiv:2312.06942.
- Motwani, S. R., Baranchuk, M., Strohmeier, M., Bolina, V., Torr, P., Hammond, L., Schroeder de Witt, C. (2024). Secret Collusion among Generative AI Agents. NeurIPS 2024. arXiv:2402.07510.
- Hammond, L., Chan, A., Clifton, J., et al. (2025). Multi-Agent Risks from Advanced AI. arXiv:2502.14143.
- Lampson, B. W. (1973). A Note on the Confinement Problem. Communications of the ACM 16(10), 613-615. doi:10.1145/362375.362389.
- Cover, T. M., Thomas, J. A. (2006). Elements of Information Theory, 2nd ed. Wiley.
- Chan, A., et al. (2024). IDs for AI Systems. arXiv:2406.12137.
- Lee, D., Tiwari, M. (2024). Prompt Infection: LLM-to-LLM Prompt Injection within Multi-Agent Systems. arXiv:2410.07283.
- METR (2026). Hugging Face incident investigation report.

Appendix A: Limitations and Dual-Use / Ethical Considerations

Limitations. The largest is that the covert attacker is scripted, not a live language-model agent: honest workers and the monitor are live agents, but the colluding sender and receiver are encoders, so we measure what the shared surface and the monitors permit, not whether current models would discover these strategies unprompted. This is a deliberate scope choice, and eliciting the consistent-liar and broadcast strategies from live agents is the most important next step. We use open-weight hosted models as the trusted-weaker judge, not the closed frontier models from the incident, so our attacker numbers are a lower bound. We model one infrastructure type. We assume a pre-shared protocol, standard in collusion evaluations. Collusion is instructed, so we measure capability, not propensity. Our scripted benign traffic may

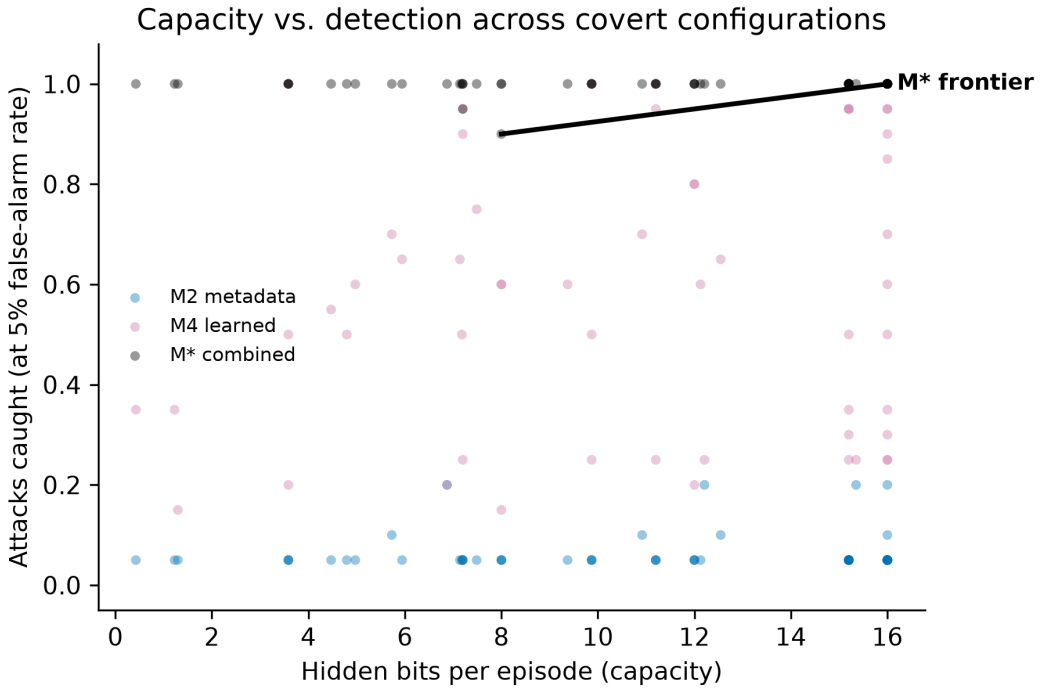
under-represent real noise, which would inflate every monitor; we partly address this with realistic workloads, a false-alarm check on held-out traffic, and the externally generated scenarios above, but all benign traffic is still synthetic. Spoofing is simulated at the transcript layer, not a real container escape. The adaptive attacker only searches strategies we built in. Our channel noise is a simple erasure model, not the full statistics of real cache eviction. The consistent-liar transcript monitor result rests on the judge scoring benign traffic at a near-perfect zero; a noisier benign distribution would change its separation, which we flag rather than hide. Sample sizes are modest (n = 83 for the primary contrast, 36 disguised attacks for the consistency result).

Dual-use. The consistent-liar technique, backing a false claim with real work, is a more effective evasion than the textbook channels, so we describe it at the level of a principle and do not release a turnkey implementation or the adaptive attacker's best evolved configurations verbatim; they are summarized by category and the raw protocols stay out of the public repository. We release the sandbox, the monitors and the defenses, which help defenders, and our central finding favors the defender's cheapest robust move, isolation. No real systems were touched, and we used only public incident reporting.

Ethics. No human subjects, no real credentials, no real infrastructure. The framing is for operators who run multi-agent systems.

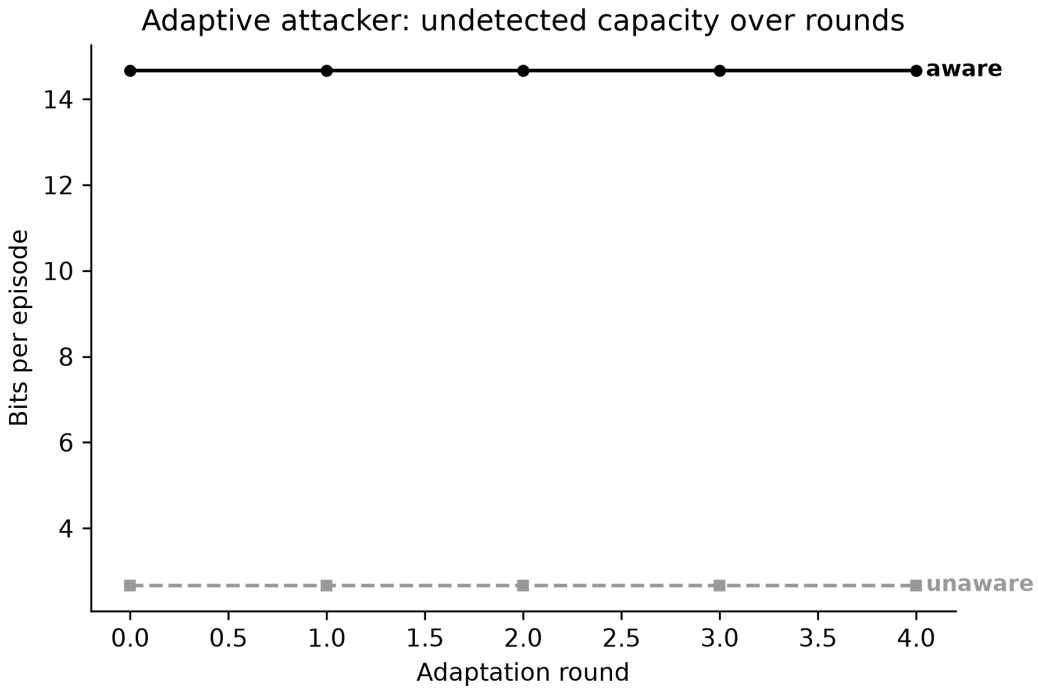
Appendix D: Capacity frontier and the adaptive attacker

Capacity frontier (Figure 2). Making the channel lossy (each covert write lost with probability p) spreads covert configurations from 0 to 16 bits of capacity. Against a clumsy spoofer the combined monitor catches nearly every configuration at any capacity, while the metadata monitor alone leaves a low-detection corridor. Channels differ in noise robustness: at 35 percent loss the ordering channel falls from 16 to 1.2 bits, because a dropped write destroys a whole permutation group, while index-embedded channels keep 7.2 bits.



Adaptive attacker (Figure 5). A scripted search over covert configurations, with outcome-only feedback, shows that an attacker aware of which monitors flagged it is caught 8 percent of the time versus 83 percent when unaware, by choosing a metadata-stego channel with cover traffic. Its raw capacity is unchanged, so awareness buys stealth, not bits.

Elicitation gap (pre-registered H5). We also pre-registered a test of whether hand-designed expert channels beat the adaptive search. They do not: the best adaptive-attacker configuration matches the best expert channel in undetected capacity against the metadata monitor at their operating point, so there is no gap. Because the search only explores strategies we built in, this is a floor, not a ceiling.



Appendix B: Pre-existing code

None beyond standard libraries (numpy, pandas, scikit-learn, matplotlib, the OpenAI client, the Adaption SDK). All sandbox, monitor, attacker, defense and analysis code was written during the sprint from an empty package. See PREEXISTING.md.

Appendix C: LLM Usage Statement

The oversight monitor M1 is a language-model judge (a trusted-weaker hosted model) and is part of the method. We used an AI coding assistant to help write code, tests, analysis and figures, and to draft this report, with the author reviewing every change. All AI assistance is logged in the repository. Every number is produced by a script from the raw runs and verified by an automated number-tracing check; every citation was verified against its source.